

Secure Cloud Infrastructure Project

AWS + Terraform | Secure architecture, implementation evidence, troubleshooting, and validation

Platform	Amazon Web Services
IaC	Terraform
Region	us-west-2
Compute	Amazon Linux 2023 / EC2
Secure administration	AWS Systems Manager Session Manager
Monitoring	Amazon CloudWatch

Project outcome

A multi-tier AWS environment was provisioned with Terraform, secured through network segmentation and IAM, administered through SSM without opening SSH, monitored with CloudWatch, and validated through direct command-line evidence.

Evidence note

Screenshots in this report were captured during the live build and troubleshooting process. AWS account identifiers and resource IDs are operational identifiers, not secret keys; redact them before public publication if desired.

1. Executive Summary

This project demonstrates a practical secure-cloud deployment rather than a minimal EC2 launch. The environment uses a dedicated VPC, segmented public and private subnets, controlled routing, security groups, Systems Manager interface endpoints, least-privilege IAM, CloudWatch alarms, and Terraform for repeatable infrastructure management.

Engineering focus

The build includes successful validation and a documented troubleshooting path: an SSM endpoint security-group rule allowed only the private subnet, causing the public instance to time out against the PrivateLink endpoint. The issue was isolated using health checks, route validation, IAM inspection, console logs, and then corrected through Terraform.

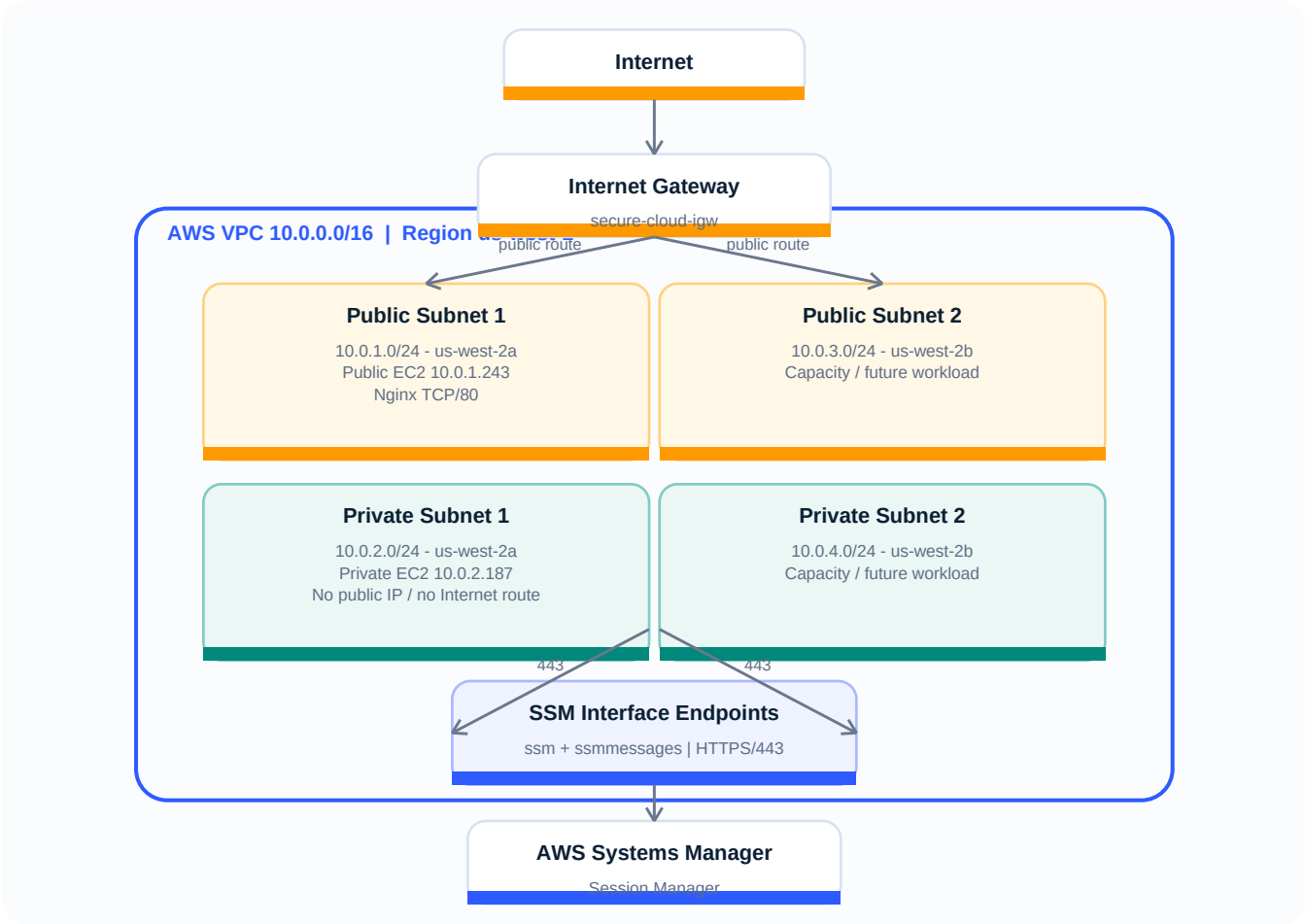
Key accomplishments

- Built a VPC (10.0.0.0/16) with two public and two private subnets across us-west-2a and us-west-2b.
- Deployed a public nginx EC2 instance and a private EC2 instance with no public IP.
- Implemented AWS Systems Manager access using an EC2 IAM role, instance profile, SSM/SSM Messages interface endpoints, and scoped Session Manager permissions.
- Validated that the private host had no general outbound Internet path while remaining manageable through SSM PrivateLink.
- Configured CloudWatch CPU alarms for both EC2 instances and added Terraform outputs for operator visibility.
- Ended with a clean Terraform plan: the live infrastructure matched the configuration.

Contents

- [2](#) Architecture
- [3](#) Terraform and deployment state
- [4](#) EC2 tier validation
- [5](#) Private-server SSM and isolation
- [6](#) IAM and Session Manager authorization
- [7](#) Public-server troubleshooting
- [8](#) Root-cause analysis
- [9](#) Infrastructure-as-Code remediation
- [10](#) Post-fix validation
- [11](#) Terraform outputs and drift check
- [12](#) CloudWatch monitoring
- [13](#) Security controls
- [14](#) Troubleshooting timeline

2. Architecture



The VPC uses segmented public and private address space. Public routing is provided by an Internet Gateway; the private route table intentionally has no default Internet route. Systems Manager access to the private tier is provided by interface endpoints for **ssm** and **ssmmessages**, keeping administration off the public Internet.

Component	Configuration / purpose
VPC	10.0.0.0/16, DNS support and hostnames enabled
Public subnet 1	10.0.1.0/24, us-west-2a; public EC2
Private subnet 1	10.0.2.0/24, us-west-2a; private EC2 and SSM endpoints
Public subnet 2	10.0.3.0/24, us-west-2b
Private subnet 2	10.0.4.0/24, us-west-2b
Public SG	Inbound HTTP/80 from Internet; outbound allowed
Private SG	Inbound TCP/8080 only from public security group
SSM endpoint SG	HTTPS/443 from 10.0.0.0/16 after remediation
Management	AWS Systems Manager Session Manager; no SSH exposure required

3. Terraform and Deployment State

Terraform was used as the source of truth for the environment. The state refresh showed the VPC, subnets, route tables, security groups, IAM role/profile, EC2 instances, SSM endpoints, and CloudWatch alarms. A clean plan confirmed that the deployed infrastructure matched the configuration.

```
C:\Users\emeka\secure-cloud-project>terraform plan
aws_iam_role.ec2_ssm_role: Refreshing state... [id=secure-cloud-ec2-ssm-role]
aws_vpc.secure_cloud_vpc: Refreshing state... [id=vpc-079d630ed26612e4e]
data.aws_ami.amazon_linux: Reading...
aws_iam_role_policy_attachment.ssm_core: Refreshing state... [id=secure-cloud-ec2-ssm-role/arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore]
aws_iam_instance_profile.ec2_ssm_profile: Refreshing state... [id=secure-cloud-ec2-ssm-profile]
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-0661cce2d6259c605]
aws_subnet.private_subnet_2: Refreshing state... [id=subnet-05cad2079edcc81f4]
aws_route_table.public_route_table: Refreshing state... [id=rtb-047aa568e22ce628f]
aws_route_table.private_route_table: Refreshing state... [id=rtb-0307e9362b6dfbcb7]
aws_subnet.public_subnet_1: Refreshing state... [id=subnet-01cacc97134409a73]
aws_subnet.public_subnet_2: Refreshing state... [id=subnet-0f888c49d662b0e0b]
aws_internet_gateway.secure_cloud_igw: Refreshing state... [id=igw-04b8dff1d33ba1186]
aws_subnet.private_subnet_1: Refreshing state... [id=subnet-0e3ac8e771b8e2a10]
aws_security_group.public_sg: Refreshing state... [id=sg-0b5966fd84883e7c5]
aws_security_group.ssm_endpoint_sg: Refreshing state... [id=sg-07123247ba7508f44]
aws_route_table_association.private_subnet_2_association: Refreshing state... [id=rtbassoc-0c545c3c0a6f53418]
aws_route.public_internet_route: Refreshing state... [id=r-rtb-047aa568e22ce628f1080289494]
aws_security_group.private_sg: Refreshing state... [id=sg-06ce62a0db92d9015]
aws_route_table_association.public_subnet_1_association: Refreshing state... [id=rtbassoc-09bfe50093d4b661]
aws_instance.public_server: Refreshing state... [id=i-09eb94f86e36bf5fa]
aws_route_table_association.public_subnet_2_association: Refreshing state... [id=rtbassoc-012ee5687e0cfc200]
aws_route_table_association.private_subnet_1_association: Refreshing state... [id=rtbassoc-0d61376138945587a]
aws_vpc_endpoint.ssm: Refreshing state... [id=vpce-07160fa031e464c1c]
aws_vpc_endpoint.ssm_messages: Refreshing state... [id=vpce-05eebd79c611c7695]
aws_instance.private_server: Refreshing state... [id=i-0a89b8e252e83dbcb]
aws_cloudwatch_metric_alarm.public_server_high_cpu: Refreshing state... [id=secure-cloud-public-high-cpu]
aws_cloudwatch_metric_alarm.private_server_high_cpu: Refreshing state... [id=secure-cloud-private-high-cpu]

No changes. Your infrastructure matches the configuration.
```

Figure 1 - Terraform plan showing no infrastructure changes.

Operational meaning

A clean plan is an important validation artifact: it demonstrates that the live AWS environment and Terraform state/configuration are aligned, reducing unmanaged configuration drift.

4. EC2 Tier Validation

The public and private instances were queried through the AWS CLI. Both were running. The public instance had a public address and a private VPC address; the private instance had only a private address.

```
C:\Users\emeka\secure-cloud-project>aws ec2 describe-instances --filters "Name=tag:Name,Values=secure-cloud-public-server,secure-cloud-private-server" --query "Reservations[].Instances[].{Name:Tags[?Key=='Name']|[0].Value,InstanceId:InstanceId,State:State.Name,PrivateIP:PrivateIpAddress,PublicIP:PublicIpAddress}" --output table
```

InstanceId	Name	PrivateIP	PublicIP
i-09eb94f86e36bf5fa	secure-cloud-public-server	10.0.1.243	35.87.133.179
i-0a89b8e252e83dbcb	secure-cloud-private-server	10.0.2.187	None

```
C:\Users\emeka\secure-cloud-project>
```

Figure 2 - EC2 instances running with public/private addressing.

Role	Instance ID	Private IP	Public IP
Public web tier	i-09eb94f86e36bf5fa	10.0.1.243	35.91.200.250 (current during final validation)
Private tier	i-0a89b8e252e83dbcb	10.0.2.187	None

5. Private-Server SSM and Internet Isolation

The private EC2 instance was deliberately configured without a public IP and without a default Internet route. Nevertheless, it remained manageable through Systems Manager because SSM traffic traversed private interface endpoints.

```
Starting session with SessionId: terraform-user-7cd7braq4fet3cqo7v8y5p2j2q
sh-5.2$ whoami
ssm-user
sh-5.2$ hostname
ip-10-0-2-187.us-west-2.compute.internal
sh-5.2$ hostname -I
10.0.2.187
sh-5.2$ cat/etc/os-release
sh: cat/etc/os-release: Not a directory
sh-5.2$ cat/etc/os-release
sh: cat/etc/os-release: Not a directory
sh-5.2$ cat /etc/os-release
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:al2023"
PRETTY_NAME="Amazon Linux 2023.12.20260817"
ANSI_COLOR="0;33"
CPE_NAME="cpe:2.3:o:amazon:amazon_linux:2023"
HOME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
BUG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
VENDOR_NAME="AWS"
VENDOR_URL="https://aws.amazon.com/"
SUPPORT_END="2029-06-30"
sh-5.2$ curl -I --max-time 10 https://aws.amazon.com
curl: (28) Connection timed out after 10002 milliseconds
sh-5.2$ |
```

Figure 3 - Private instance validation: Amazon Linux 2023 and outbound Internet test timing out.

Security result

The private server was reachable through Session Manager while a direct HTTPS request to the public Internet timed out. This demonstrates administrative access without exposing the host to the Internet or opening inbound SSH.

Validated properties

- Session user: ssm-user
- Private IP: 10.0.2.187
- Operating system: Amazon Linux 2023
- No public IP assigned
- No general outbound Internet route
- SSM connectivity preserved through VPC interface endpoints

6. IAM and Session Manager Authorization

The EC2 instances use the **secure-cloud-ec2-ssm-role** through the **secure-cloud-ec2-ssm-profile**. The role is trusted by EC2 and has the AWS-managed **AmazonSSMManagedInstanceCore** policy. The human/CLI identity used for Terraform was intentionally not given blanket SSM shell access.

```
C:\Users\emeka\secure-cloud-project>aws ssm start-session --target i-0a89b8e252e83dbc  
cb  
  
aws: [ERROR]: An error occurred (AccessDeniedException) when calling the StartSession  
n operation: User: arn:aws:iam::572059938781:user/terraform-user is not authorized t  
o perform: ssm:StartSession on resource: arn:aws:ssm:us-west-2:572059938781:document  
/SSM-SessionManagerRunShell because no identity-based policy allows the ssm:StartSes  
sion action  
  
C:\Users\emeka\secure-cloud-project>
```

Figure 4 - Initial ssm:StartSession denial for terraform-user.

The denial was expected from a least-privilege perspective: the user could provision and inspect infrastructure but could not automatically open an interactive shell. A scoped Session Manager policy was added to allow StartSession to the intended EC2 resources and the standard SSM session document, plus ResumeSession/TerminateSession and the required messaging channel action.

Control separation

EC2 instance permissions and operator permissions are separate. The instance role lets the SSM Agent communicate with Systems Manager; the operator policy determines whether a human identity may start a session.

7. Public-Server Troubleshooting

The public nginx server remained healthy and reachable over HTTP, but its SSM status showed **ConnectionLost**. This separation was useful: the application/data plane was functioning while the management plane was not.

```
C:\Users\emeka\secure-cloud-project>aws ssm describe-instance-information --query "InstanceInformationList[?InstanceId=='i-09eb94f86e36bf5fa'].{InstanceId:InstanceId,PingStatus:PingStatus,LastPing:LastPingDateTime,AgentVersion:AgentVersion}" --output table
```

DescribeInstanceInformation	
AgentVersion	3.3.4624.0
InstanceId	i-09eb94f86e36bf5fa
LastPing	2026-09-04T21:27:18.535000-07:00
PingStatus	ConnectionLost

```
C:\Users\emeka\secure-cloud-project>curl http://35.87.133.179
<h1>Secure Cloud Public Server</h1>

C:\Users\emeka\secure-cloud-project>
```

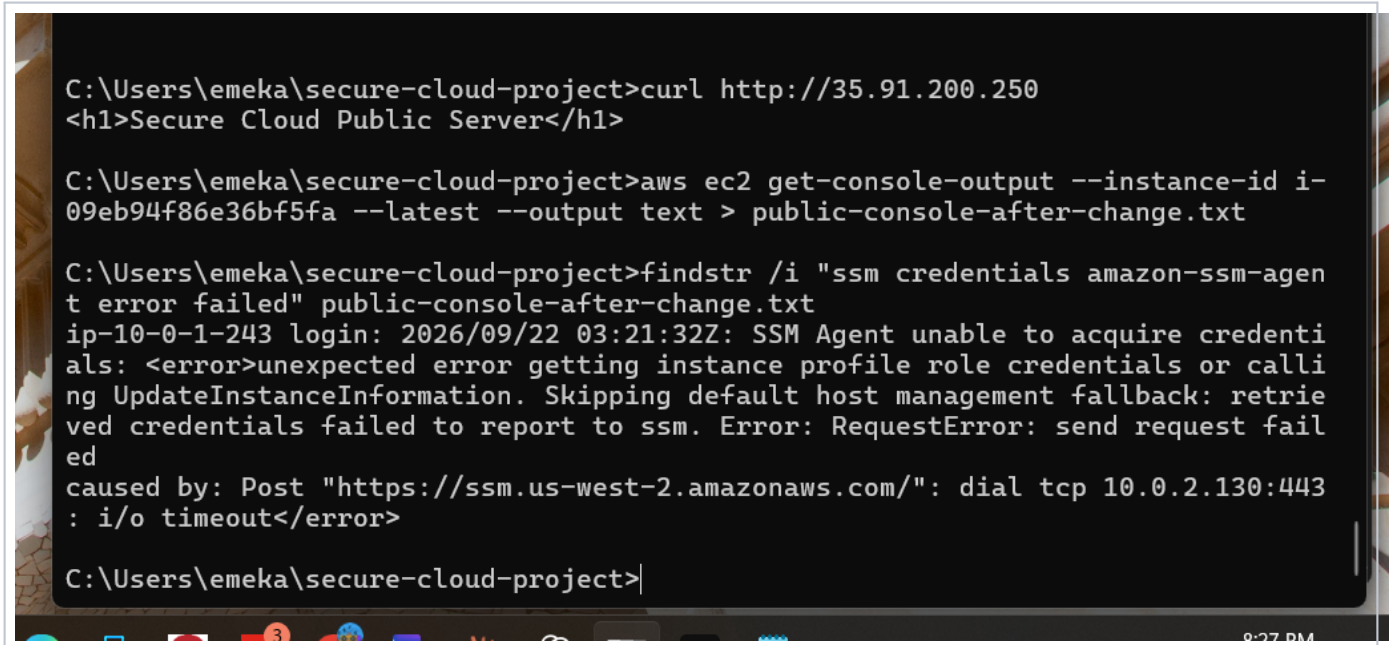
Figure 5 - Public HTTP response succeeded while SSM still reported a lost connection.

A structured differential diagnosis was performed rather than immediately recreating the host. The following were verified in sequence:

- EC2 instance running; system and instance status checks OK.
- Public route table contained 0.0.0.0/0 to the Internet Gateway.
- Public security group allowed required HTTP and outbound traffic.
- IAM instance profile was attached to the public instance.
- Instance profile contained secure-cloud-ec2-ssm-role.
- AmazonSSMManagedInstanceCore was attached to the role.
- IMDSv2 was enabled and required.
- SSM Agent was present on the instance.

8. Root Cause: SSM Endpoint Security Group

Console output exposed the decisive error. The SSM Agent attempted to call the regional SSM hostname, which private DNS resolved to the VPC interface endpoint. The TCP connection to the endpoint timed out on port 443.



```
C:\Users\emeka\secure-cloud-project>curl http://35.91.200.250
<h1>Secure Cloud Public Server</h1>

C:\Users\emeka\secure-cloud-project>aws ec2 get-console-output --instance-id i-09eb94f86e36bf5fa --latest --output text > public-console-after-change.txt

C:\Users\emeka\secure-cloud-project>findstr /i "ssm credentials amazon-ssm-agent error failed" public-console-after-change.txt
ip-10-0-1-243 login: 2026/09/22 03:21:32Z: SSM Agent unable to acquire credentials: <error>unexpected error getting instance profile role credentials or calling UpdateInstanceInformation. Skipping default host management fallback: retrieved credentials failed to report to ssm. Error: RequestError: send request failed
caused by: Post "https://ssm.us-west-2.amazonaws.com/": dial tcp 10.0.2.130:443 : i/o timeout</error>

C:\Users\emeka\secure-cloud-project>
```

Figure 6 - SSM Agent timeout to the private interface endpoint on TCP/443.

Root cause

The endpoint security group allowed HTTPS only from 10.0.2.0/24 (the private subnet). The public EC2 instance lived at 10.0.1.243, so it could resolve the SSM service to the interface endpoint but could not pass the endpoint security-group ingress rule.

This explained the asymmetric behavior: the private host could use SSM because it was inside the allowed 10.0.2.0/24 range, while the public host was blocked despite otherwise healthy networking and IAM.

9. Infrastructure-as-Code Remediation

The SSM endpoint security-group ingress rule was corrected in Terraform. The source CIDR for TCP/443 was expanded from the single private subnet to the VPC CIDR, 10.0.0.0/16. This did not expose the endpoint to the Internet; it allowed HTTPS only from addresses inside the VPC.

```
Terraform will perform the following actions:

# aws_security_group.ssm_endpoint_sg will be updated in-place
~ resource "aws_security_group" "ssm_endpoint_sg" {
  id          = "sg-07123247ba7508f44"
  ~ ingress   = [
    - {
      - cidr_blocks      = [
        - "10.0.2.0/24",
      ]
      - description      = "HTTPS from private subnet"
      - from_port         = 443
      - ipv6_cidr_blocks = []
      - prefix_list_ids   = []
      - protocol          = "tcp"
      - security_groups   = []
      - self              = false
      - to_port           = 443
    },
    + {
      + cidr_blocks      = [
        + "10.0.0.0/16",
      ]
      + description      = "HTTPS from VPC instances to SSM endpoints"
      + from_port         = 443
      + ipv6_cidr_blocks = []
      + prefix_list_ids   = []
      + protocol          = "tcp"
      + security_groups   = []
      + self              = false
      + to_port           = 443
    },
  ]
  name        = "secure-cloud-ssm-endpoint-sg"
  tags        = {
    "Environment" = "dev"
    "Name"         = "secure-cloud-ssm-endpoint-sg"
    "Project"      = "SecureCloudProject"
  }
  # (9 unchanged attributes hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.
```

Figure 7 - Terraform plan changing SSM endpoint ingress from 10.0.2.0/24 to 10.0.0.0/16.

Terraform proposed an in-place security-group update with no resource destruction. The change was applied and the public instance rebooted so the SSM Agent could establish a fresh connection through the now-permitted endpoint path.

Why Terraform mattered

The fix was encoded in configuration instead of performed as an undocumented console-only change. Future deployments inherit the corrected rule, preserving repeatability and auditability.

10. Post-Fix Validation

After the endpoint security-group correction, both EC2 instances reported **Online** in Systems Manager.

```
9eb94f86e36bf5fa

C:\Users\emeka\secure-cloud-project>aws ec2 describe-instance-status --instance-ids i-09eb94f86e36bf5fa --include-all-instances --query "InstanceStatuses[0].{State:InstanceState.Name,System:SystemStatus.Status,Instance:InstanceStatus.Status}" --output table
-----
| DescribeInstanceStatus |
+-----+-----+-----+
| Instance | State | System |
+-----+-----+-----+
| ok | running | ok |
+-----+-----+-----+

C:\Users\emeka\secure-cloud-project>aws ssm describe-instance-information --query "InstanceInformationList[0].{InstanceId:InstanceId,PingStatus:PingStatus,LastPing:LastPingDateTime}" --output table
-----
| DescribeInstanceInformation |
+-----+-----+-----+
| InstanceId | LastPing | PingStatus |
+-----+-----+-----+
| i-09eb94f86e36bf5fa | 2026-09-21T20:33:50.033000-07:00 | Online |
| i-0a89b8e252e83dbcb | 2026-09-21T20:30:55.017000-07:00 | Online |
+-----+-----+-----+

C:\Users\emeka\secure-cloud-project>
```

Figure 8 - Both public and private EC2 instances online in Systems Manager.

```
C:\Users\emeka\secure-cloud-project>aws ssm start-session --target i-09eb94f86e36bf5fa

aws: [ERROR]: An error occurred (AccessDeniedException) when calling the StartSession operation: User: arn:aws:iam::572059938781:user/terraform-user is not authorized to perform: ssm:StartSession on resource: arn:aws:ec2:us-west-2:572059938781:instance/i-09eb94f86e36bf5fa because no identity-based policy allows the ssm:StartSession action

C:\Users\emeka\secure-cloud-project>aws ssm start-session --target i-09eb94f86e36bf5fa

Starting session with SessionId: terraform-user-vl67bfve5nfg3hb5fpcezv3dpa
sh-5.2$ whoami
hossm-user
sh-5.2$ hostname -I
10.0.1.243
sh-5.2$ systemctl is-active amazon-ssm-agent
systactive
sh-5.2$ systemctl is-active nginx
active
sh-5.2$ exit
exit

Exiting session with sessionId: terraform-user-vl67bfve5nfg3hb5fpcezv3dpa.

C:\Users\emeka\secure-cloud-project>
```

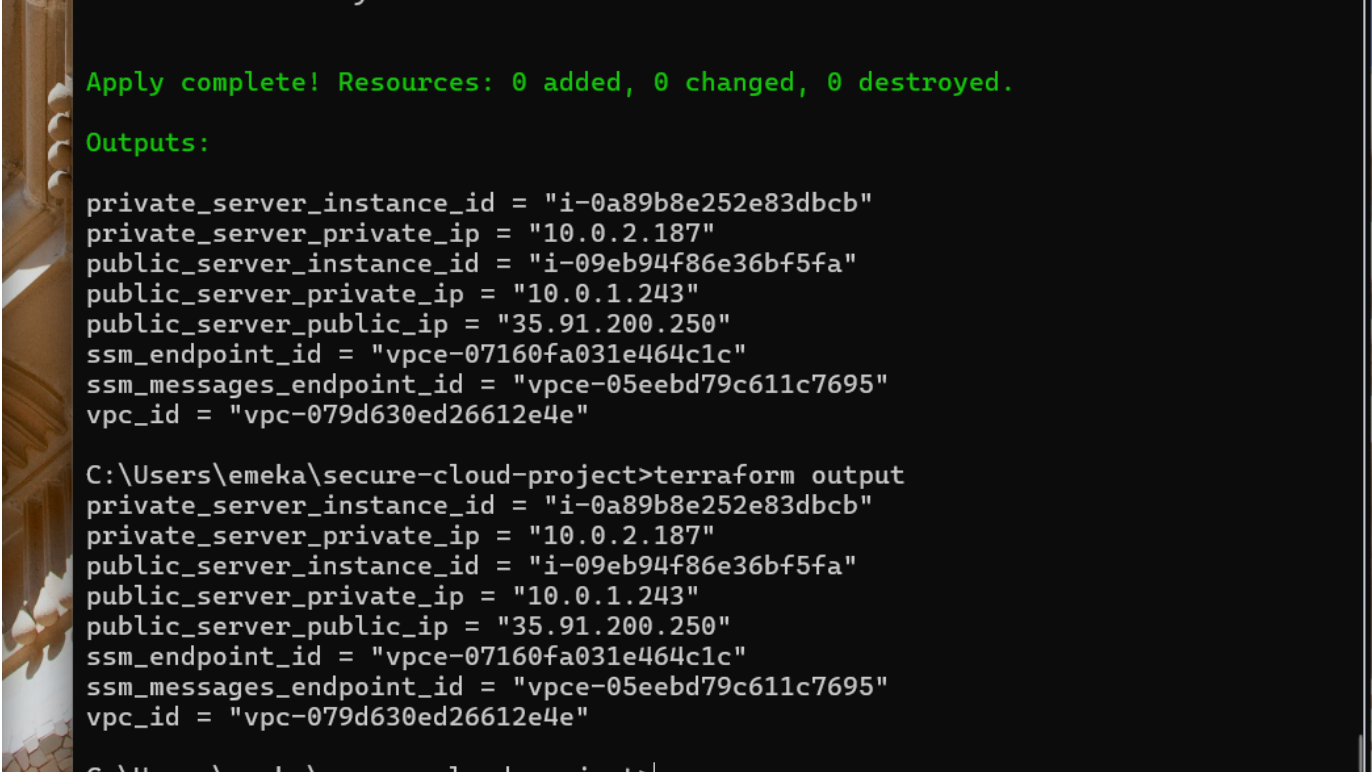
Figure 9 - Public EC2 Session Manager validation: ssm-user, 10.0.1.243, SSM Agent active, nginx active.

Validation outcome

The public server was manageable through Session Manager and continued to serve nginx. The private server remained isolated from the public Internet but manageable through the same SSM control plane.

11. Terraform Outputs and Drift Check

An outputs.tf file was added so operators can retrieve the key deployed identifiers without manual AWS queries. Outputs included the VPC ID, public/private instance IDs, IP addresses, and SSM endpoint IDs.

A screenshot of a terminal window with a dark background. The text is displayed in a light green color. The first line shows a successful Terraform apply command: "Apply complete! Resources: 0 added, 0 changed, 0 destroyed." This is followed by a section header "Outputs:". Below this, a list of Terraform outputs is shown, each with a key and a value in quotes. The keys are: private_server_instance_id, private_server_private_ip, public_server_instance_id, public_server_private_ip, public_server_public_ip, ssm_endpoint_id, ssm_messages_endpoint_id, and vpc_id. The values are AWS resource identifiers and IP addresses. At the bottom, a command prompt shows the user running "terraform output" to view these values again.

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
private_server_instance_id = "i-0a89b8e252e83dbcb"  
private_server_private_ip = "10.0.2.187"  
public_server_instance_id = "i-09eb94f86e36bf5fa"  
public_server_private_ip = "10.0.1.243"  
public_server_public_ip = "35.91.200.250"  
ssm_endpoint_id = "vpce-07160fa031e464c1c"  
ssm_messages_endpoint_id = "vpce-05eebd79c611c7695"  
vpc_id = "vpc-079d630ed26612e4e"  
  
C:\Users\emeka\secure-cloud-project>terraform output  
private_server_instance_id = "i-0a89b8e252e83dbcb"  
private_server_private_ip = "10.0.2.187"  
public_server_instance_id = "i-09eb94f86e36bf5fa"  
public_server_private_ip = "10.0.1.243"  
public_server_public_ip = "35.91.200.250"  
ssm_endpoint_id = "vpce-07160fa031e464c1c"  
ssm_messages_endpoint_id = "vpce-05eebd79c611c7695"  
vpc_id = "vpc-079d630ed26612e4e"  
  
C:\Users\emeka\secure-cloud-project>
```

Figure 10 - Terraform outputs after applying output-only state changes.

```

ec2_ssm_project
aws_iam_role_policy_attachment.ssm_core: Refreshing state... [id=secure-cloud-e
c2-ssm-role/arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore]
data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0661cce2d6259c605]
aws_internet_gateway.secure_cloud_igw: Refreshing state... [id=igw-04b8dff1d33b
a1186]
aws_subnet.public_subnet_2: Refreshing state... [id=subnet-0f888c49d662b0e0b]
aws_security_group.public_sg: Refreshing state... [id=sg-0b5966fd84883e7c5]
aws_subnet.private_subnet_1: Refreshing state... [id=subnet-0e3ac8e771b8e2a10]
aws_route_table.private_route_table: Refreshing state... [id=rtb-0307e9362b6d6b
cb7]
aws_security_group.ssm_endpoint_sg: Refreshing state... [id=sg-07123247ba7508f4
4]
aws_subnet.private_subnet_2: Refreshing state... [id=subnet-05cad2079edcc81f4]
aws_route_table.public_route_table: Refreshing state... [id=rtb-047aa568e22ce62
8f]
aws_subnet.public_subnet_1: Refreshing state... [id=subnet-01cacc97134409a73]
aws_route_table_association.private_subnet_2_association: Refreshing state... [
id=rtbassoc-0c545c3c0a6f53418]
aws_route_table_association.private_subnet_1_association: Refreshing state... [
id=rtbassoc-0d61376138945587a]
aws_security_group.private_sg: Refreshing state... [id=sg-06ce62a0db92d9015]
aws_instance.public_server: Refreshing state... [id=i-09eb94f86e36bf5fa]
aws_vpc_endpoint.ssm: Refreshing state... [id=vpce-07160fa031e464c1c]
aws_vpc_endpoint.ssm_messages: Refreshing state... [id=vpce-05eebd79c611c7695]
aws_route_table_association.public_subnet_1_association: Refreshing state... [i
d=rtbassoc-09bfe50093d4b661]
aws_route_table_association.public_subnet_2_association: Refreshing state... [i
d=rtbassoc-012ee5687e0cfc200]
aws_route.public_internet_route: Refreshing state... [id=r-rtb-047aa568e22ce628
f1080289494]
aws_instance.private_server: Refreshing state... [id=i-0a89b8e252e83dbcb]
aws_cloudwatch_metric_alarm.public_server_high_cpu: Refreshing state... [id=sec
ure-cloud-public-high-cpu]
aws_cloudwatch_metric_alarm.private_server_high_cpu: Refreshing state... [id=se
cure-cloud-private-high-cpu]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.

C:\Users\emeka\secure-cloud-project>

```

Figure 11 - Final Terraform plan: no changes; infrastructure matches configuration.

A test was also performed to replace the pinned AMI with a dynamic most-recent AMI data source. Terraform correctly proposed replacement of both EC2 instances, so the change was intentionally reverted to avoid unnecessary churn. The tested AMI remains pinned for controlled upgrades and reproducibility.

12. CloudWatch Monitoring

Two CloudWatch alarms were validated: one for the public EC2 instance and one for the private EC2 instance. Both monitor EC2 CPUUtilization and were in the OK state during final verification.

```
C:\Users\emeka\secure-cloud-project>aws cloudwatch describe-alarms --alarm-names secure-cloud-public-high-cpu secure-cloud-private-high-cpu --query "MetricAlarms[0].{Name:AlarmName,State:StateValue,Threshold:Threshold,Period:Period,EvaluationPeriods:EvaluationPeriods,Instance:Dimensions[0].Value}" --output table
-----
|                                     DescribeAlarms                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| EvaluationPeriods | Instance | Name |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Period | State | Threshold |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | OK | 80.0 | i-0a89b8e252e83dbcb | secure-cloud-private-high-cpu |
| 300 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | OK | 80.0 | i-09eb94f86e36bf5fa | secure-cloud-public-high-cpu |
| 300 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
C:\Users\emeka\secure-cloud-project>
```

Figure 12 - CloudWatch alarm validation for both EC2 instances.

Alarm	Target	Threshold	Period	Evaluation periods	Final state
secure-cloud-public-high-cpu	Public EC2	80%	300 sec	2	OK
secure-cloud-private-high-cpu	Private EC2	80%	300 sec	2	OK

13. Security Controls Demonstrated

Control	Implementation	Security value
Network segmentation	Public and private subnets with separate route tables	Limits exposure and separates Internet-facing from internal workloads.
No public IP on private host	Private EC2 receives only 10.0.2.187	Prevents direct Internet reachability.
No SSH exposure required	Administration performed through SSM Session Manager	Avoids opening port 22 and reduces key-management burden.
Least-privilege IAM	Separate EC2 role and operator Session Manager permissions	Separates machine permissions from human/operator permissions.
Private AWS service access	SSM and ssmmessages interface endpoints	Allows management traffic to stay inside AWS networking.
Security-group isolation	Private application port 8080 only from public SG	Restricts east-west application access to the intended source tier.
Monitoring	CloudWatch CPU alarms on both instances	Provides basic operational visibility and a trigger for abnormal CPU behavior.
IaC governance	Terraform plan/apply and final no-change verification	Makes changes reviewable and reduces configuration drift.

Public-sharing hygiene

This public case study contains no access keys, secret keys, session tokens, or passwords. AWS resource identifiers shown in screenshots are project metadata rather than authentication secrets; they can still be redacted if a stricter publication policy is preferred. Never publish access keys, secret keys, session tokens, or credentials.

14. Troubleshooting Timeline

Stage	Observed symptom	Evidence / decision	Resolution
1. IAM provisioning	Earlier IAM role creation was denied	Terraform user lacked required IAM role-management action	Least-privilege IAM policy was expanded only for required project resources.
2. Private SSM access	StartSession denied	terraform-user lacked ssm:StartSession	Scoped Session Manager operator permissions were added.
3. Local tooling	SessionManagerPlugin not found	AWS CLI could authorize but local plugin was missing	Installed Amazon Session Manager Plugin on Windows.
4. Private validation	Private SSM online; Internet curl timed out	10.0.2.187 reachable through SSM, no general Internet path	Confirmed intended isolation + PrivateLink management.
5. Public management	Public server HTTP healthy, SSM ConnectionLost	Health, routing, IAM, IMDS, and agent presence were verified	Narrowed fault to SSM network path.
6. Root cause	SSM endpoint connection timed out on 10.0.2.x:443	Endpoint SG allowed only 10.0.2.0/24	Changed endpoint SG source to VPC CIDR 10.0.0.0/16 in Terraform.
7. Final validation	Both instances reported Online	Session Manager shell opened successfully to public and private instances	Management plane validated end to end.